Quantum classical hybrid neural networks for continuous variable prediction

Prateek Jain[1,2] and Alberto Garcia Garcia[2,3]

[1]Fractal Analytics, Gurgaon, India
[2]Universidad Politécnica de Madrid, Spain
[3]Accenture, Madrid, Spain

arXiv:2212.04209v5 [quant-ph] 9 Mar 2023

Within this decade, quantum computers are predicted to outperform conventional computers in terms of processing power and have a disruptive effect on a variety of business sectors. It is predicted that the financial sector would be one of the first to benefit from quantum computing both in the short and long terms. In this research work we use Hybrid Quantum Neural networks to present a quantum machine learning approach for Continuous variable prediction.

1 Introduction

Quantum Machine Learning is an emerging field that shows promising approaches to solve intractable problems. There are quite a few quantum machine learning algorithms that have emerged from their classical counterparts for example the so called Quantum Neural Networks. Quantum neural networks are variational quantum circuits that store quantum information in continuous degrees of freedom, like the amplitudes of an electromagnetic field. Multiple layers of continuously parameterized gates, which are essential to quantum processing, makes up this circuit [1].

The main goal of this research work is to develop a somewhat clearer understanding of the promises and limitations of the current quantum algorithms for machine learning and define some future directions for research. In this work, we will use a parameterized quantum circuit as one of the layer of a Hybrid Quantum Neural Network to present usage for financial applications. The work focuses to demonstrate the use of Hybrid QNNs for Continuous variable predictions for example Asset price prediction in this case. We use the Boston housing data taken from the StatLib library maintained at Carnegie Mellon University for training the model.

2 Quantum ML, preliminaries

In last two decades due to increased computational power and the availability of vast amounts of data, Machine Learning & Deep Learning [2] in particular has seen an immense success with applications ranging from computer vision [3] to Natural Speech Synthesis [4], playing complex games like Atari & Go [5]. However, over past few years, challenges have surfaced that threaten to slow down this revolution. These challenges are, increasingly overwhelming size of the available data sets & nearing end of Moore's law. While novel developments in hardware architectures, such as graphics processing units GPUs or tensor processing units TPUs, enable orders of magnitude of improved performance compared to central processing units CPUs, they still can barely cope up much with increasing computational needs.

On the other hand, a new technological paradigm Quantum computing, term first popularised by famous physicist Richard Feynman [6] [7] a form of computation that makes use of quantum-mechanical phenomena such as superposition & entanglement [8], is predicted to overcome these limitations in classical computers. Quantum algorithms, have been investigated since 1980s [9]. In recent years one area that has received particular attention is quantum machine learning [10] the interplay of quantum mechanics and machine learning.

2.1 Quantum Neural Networks (QNN)

Parameterized quantum circuits (PQC) [11] are quantum circuits which are primarily made with a combination of fixed gates like CNOT gates & adjustable gates like Pauli rotations [12]. The adjustable parts of the circuit are parameterized, it is these parameters which are optimized to converge to an optimal solution or expected value. Quantum processors are used to evaluate the circuits, while the parameters are optimized using various loss function evaluation techniques like gradient descent on a classical computer. This hybrid approach is much less demanding in terms of number of qubits and the number of layers required in the quantum circuit hence this hybrid approach is much more suitable for NISQ era [13].

Lately, there has been a lot of research on the use of PQC as machine learning models, also commonly termed as Quantum Neural Network (QNN) outlined by Farhi and Neven [14] and comprehensive compar-

Prateek Jain: pratjz@gmail.com

ison Classical NN vs Quantum NN done by Abbas et al [15].

QNNs can attribute their origin in discussions for the essential role which quantum processes play in human brain. For example, Roger Penrose has argued that a new physics binding quantum phenomena with general relativity can explain such mental abilities as understanding, awareness and consciousness [16].

In general, typical structure of QNNs can be broadly broken into three stages: feature encoding, processing and measurement with potential post-processing. This general procedure is summarized in Fig. 1

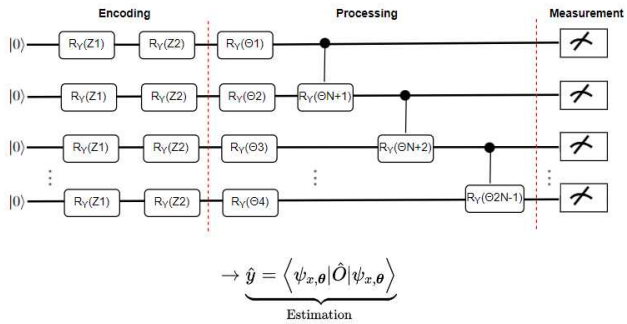

Figure 1: General architecture of a QNN.

The architecture consists of three key steps:

1. Encode the feature vector x to n-qubits by using a unitary transformation

$$|\psi_x\rangle = U_{\phi(x)}|0\rangle$$

2. Next, a circuit parameterized by parameters $\theta = (\theta 1, \cdots, \theta \mathrm{n})$ is applied which is equivalent to the following initial state

$$|\Psi_{x_a\theta}\rangle = U_\theta \, |\psi_x\rangle$$

This circuit U_θ is commonly called an ansätze & serves as a general way of transforming the state $|\psi_x\rangle$ by encoding the data in the Hilbert space.

3. Lastly the measurements of the circuit would lead to estimation of the expectation value of the observable $\hat{O}$ this state will be our model output.

$$\hat{y} = f_{\cos}(x;\theta) = \left\langle \psi_{x_0\theta} | \hat{O} | \psi_{x_n\theta} \right\rangle$$

Now we would need to optimize this output to get suitable results & thus we will need to adjust the parameters ...θn to minimize some loss function for example

$$L(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} L\left(\hat{y}_i, y_i\right)$$

In the following sections, we will briefly look at few options of feature encoding, circuit optimization & loss landscape

2.2 Feature Encoding

The very first step to initialize a circuit is feature encoding which encodes classical data x into quantum data $|\psi(x\rangle = U(x)|0\rangle \otimes n$ using a parameterized quantum circuit. There are mainly three ways of doing feature encoding i.e. Qubit encoding, Angle encoding & Amplitude encoding [17].

2.2.1 Basis Encoding

Also called as Qubit encoding in this data encoded in qubits as it is encoded in bits thus it is the simplest method, let the classical dataset be data $= \{x_1, x_2, \ldots, x_N\}$ where each data point x_i has f feature variables, let each data point x_i be denoted by a unique bitstring $\vec{b^i} = b_0^i b_1^i \ldots b_{l-1}^i$, where l depends on the data type, if it is 32-bit floating point number then $l = 32\mathrm{f}$ for f features, then the data point x_i can be encoded in quantum register consisting of l qubits:

$$x_i \mapsto \left|b_0^i\right\rangle \left|b_1^i\right\rangle \ldots \left|b_{l-1}^i\right\rangle \equiv \left|\vec{b^i}\right\rangle.$$

for example we have 3 features as $x_1 = 1$, $x_2 = 2$, $x_2 = 3$ their binary representations would be $x_1 = 001$, $x_2 = 010$, $x_2 = 011$ thus corresponding basis encoding uses 3 qubits as $|x_1\rangle = |001\rangle$, $|x_2\rangle = |010\rangle$, $|x_2\rangle = |011\rangle$ resulting in encoded state

$$|D\rangle = \frac{1}{\sqrt{3}} \left(|001\rangle + |010\rangle + |011\rangle\right)$$

For low depth circuits qubit encoding is suitable, But the state resulting from qubit encoding is very simple mathematically, which might limit the expressive power of the model.

2.2.2 Angle Encoding

In angle encoding each feature variable is encoded to a qubit hence will need n qubits, for n features. The encoding is done by performing a Pauli-rotation on each qubit with a rotational angle equal to the corresponding feature.

With Rz gate a Hadamard gate is used on each qubit to create a superposition else these rotations would leave $|0\rangle$ invariant. For example, two features $x = (x_1, x_2)$ can be qubit encoded onto two qubits in the following way using Ry rotations:

$$R_y\left(x_1\right) \otimes R_y\left(x_2\right)\left(|0\rangle \otimes |0\rangle\right) =$$

$$\left(\cos\left(\frac{x_1}{2}\right)|0\rangle + \sin\left(\frac{x_1}{2}\right)|1\rangle\right) \otimes \left(\cos\left(\frac{x_2}{2}\right)|0\rangle + \sin\left(\frac{x_2}{2}\right)|1\rangle\right)$$

Angle encoding produces much more complex feature maps which can be made more complex by repeating the encoding process multiple times. The number of these repeated layers is called the depth of the feature map. However, angle encoding is also much more computationally expensive, since it requires circuit depth of $O(n^2)$ for n features and fully connected qubits.

2.2.3 Amplitude Encoding

In Amplitude encoding features encoded as amplitudes of a quantum state. Given a data set $x = (x_1, ..., x_N)$, where $N = 2^n$, amplitude encoding involves preparing a state

$$|\psi_{\boldsymbol{x}}\rangle = \sum_{i=1}^{N} x_i |i\rangle$$

on n qubits. Because of the normalization of the quantum states, it is essential to scale the data to make sure that sum of probabilities i.e. $\sum_{i=1}^{N} |x_i|^2 = 1$ holds.

for example let say we have a datapoint x with features $(1, 0, 6.8, 1.0)$ then $x_{norm} = \frac{1}{\sqrt{48.24}}(1, 0, 6.8, 1.0)$, the corresponding amplitude encoding uses two qubits resulting in encoded state as

$$|\psi_{x_{norm}}\rangle = \frac{1}{\sqrt{48.24}} (1|00\rangle + 0|01\rangle + 6.8|10\rangle + 1.0|10\rangle)$$

$$= \frac{1}{\sqrt{48.24}} (1|00\rangle + 6.8|10\rangle + 1.0|10\rangle)$$

The key advantage of amplitude encoding is that the number of amplitudes is pretty much limitless in the number of qubits therefore only $log_2(n)$ qubits are needed to encode n features thus an enormous amount of information can be encoded with each qubit added. However, there is not much efficient way of preparing the state.

2.3 Optimization of a PQC/QNN

A key step for hybrid PQCs is the optimization of the parameters θ for the initial ansatz. These parameters are optimized with respect to an objective function specific to a given problem. There are multiple methods for optimizing PQCs, one example of such method is numerical differentiation of the loss function:

$$\frac{\partial}{\partial \theta_i} L(\theta) \approx \frac{L(\theta_1, \cdots, \theta_i + \epsilon, \cdots \theta_{n_\theta}) - L(\theta_1, \cdots, \theta_i, \cdots \theta_{n_\theta})}{\epsilon}$$

One can optimize the parameters using techniques similar to gradient descent.

2.4 Barren Plateaus in Loss Landscape

While recent researches have shown multiple promising characteristics of QNNs, like faster training & better generalizeability but these studies have been largely focused on smaller systems. McClean et al [18]. established results relating the magnitude of the gradient to the number of qubits. They found that the variance of the expected value of randomly initialized PQCs vanish exponentially with increasing number of qubits & circuit depth.

The vanishing of PQCs gradients manifests itself as loss landscapes that are extremely flat in most of parameter space, hence the name Barren Plateaus, similar to the vanishing gradient phenomenon of classical neural networks as the depth increases. This is currently a very active research area few techniques have been proposed like each layerwise learning of gradient for the parameters [19]

To evaluate an ansatz design different techniques & circuit descriptors are used such as the circuit expressability which is the efficiency with which a quantum circuit may exploit the Hilbert Space, and entanglement capability which quantifies a circuit's capability of detecting correlations among features.

2.5 Expressability

A circuit is considered to be expressive if it is able to generate pure quantum states that are a good representation of the Hilbert space in consideration. Sim et al (2019)'s [20] method compares the ensemble of Haar random states to distribution of states read by sampling a Circuit's(PQC's) parameters to calculate expressability they proposed to approximate the distribution of fidelities as the overlap of states defined as:

$$F = \langle \psi_\theta \psi_\phi \mid \psi_\theta \psi_\phi \rangle^2$$

For the ensemble of Haar random states the probability density function of fidelities is defined as:

$$P_{\text{Harr}}(F) = (N-1)(1-F)^{N-1},$$

where F corresponds to the fidelity and N is the dimension of the Hilbert space [21]. After collecting sufficient samples of the state fidelities, the Kullback-Leibler (KL) divergence [22], between the estimated fidelity distribution and that of the Haar distributed ensemble can be computed to define expressibility as:

$$\text{Expr} = D_{KL}\left(\left(\hat{P}_{PQC}(F;\theta) \| P_{\text{Haar}}(F)\right)\right),$$

lower the KL divergence higher is the Expressability of the circuit.

2.6 Entangling Capability

A parametrized quantum circuit with lesser number of layers which is able to generate highly entangled states, is said to have better advantage to capture nontrivial correlation in quantum data Schuld and Petruccione [10]. Strongly entangled circuits can be created by repeating circuit layers made of various two-qubit parameterized gates like CNOT, CZ. Sim et al (2019) [20] proposed using Meyer-Wallach (MW) [23] entanglement measure Q to approximate entangling capability of a PQC. For a given PQC the entangling capability(Ent) can be estimated by sampling

the circuit parameters and calculating the average of the MW measure Q of output states defined as:

$$Ent = \frac{1}{S} \sum_{\theta_i \in S} Q\left(\psi_{\theta_i}\right),$$

where S denotes the set of sampled circuit parameter vector θ.

2.7 QML for Finance Applications

A myriad of problems in finance industry are addressed using Machine learning & deep Learning techniques but there are still a lot of cases where classical computation techniques turn out to be inadequate for example NP-hard problems like portfolio optimization [24], currency arbitration [25] etc..There have been a lot of areas identified [26] where-in proposed QML techniques could be used to address such problems. For this study we chose asset pricing use case wherein we use QML techniques to predict property prices.

3 Frameworks & libraries

3.1 PennyLane

Is an open-source framework built around quantum differentiable programming by Xanadu implemented in Python to achieve machine learning tasks with quantum computers [27]. It Supports hybrid quantum and classical models allowing users to connect quantum hardware with PyTorch, TensorFlow, Qiskit, Cirq etc..therefore it is Hardware agnostic i.e. same quantum circuit model can execute on different backends and allows plugins for access to diverse devices, including Strawberry Fields, Amazon Braket, IBM Q, Google Cirq, Rigetti Forest, Microsoft QDK..

3.2 StrawberryFields

Strawberry Fields is an open-source framework for photonic quantum computing again by Xanadu [28]. In particular, Strawberry Fields allows to Construct and simulate continuous-variable quantum photonic circuits. Provided simulators include highly optimized Gaussian, Fock, and Bosonic numeric backends and a TensorFlow backend for backpropagation.

Compared to qubit-based systems, photonic quantum programs use a different gate set, and have different near-term applications. Strawberry Fields is a full-stack solution for constructing, compiling, simulating, optimizing, and executing photonic algorithms.

3.3 Tensorflow Quantum & Keras

is another framework by google it brings the power of Tensorflow to the quantum world[29]. It provides high-level abstractions for the design and training of QNNs including discriminative and generative quantum models under TensorFlow and supports high-performance quantum circuit simulators along with high level wrapers for Keras [30].

Combination of all these three libraries were used in this work to create Hybrid Quantum Neural networks for predicting property prices from the Boston Housing dataset.

4 Methodology and Setup

In this section, we will go through details of implementation of algorithms and architecture designed. The framework is also capable of implementing hybrid models mixing both DNN and QNN layers.

4.1 DataSet

We use Boston Housing Data it was originally hosted on UCI Machine Learning Repository but now can be directly accessed from keras.datasets. Data Samples contain 13 feature variables of houses at different locations around the Boston suburbs in the late 1970s. Target or predictor variable is the median price value of the houses (in k$). The dataset has total 506 samples. Following is description of the features: .

CRIM - per capita crime rate by town
ZN - proportion of residential land zoned for lots over 25,000 sq.ft
INDUS - proportion of non-retail business acres per town
CHAS - Charles River dummy variable (= 1 if tract bounds river; 0 otherwise)
NOX - nitric oxides concentration (parts per 10 million)
RM - average number of rooms per dwelling
AGE - proportion of owner-occupied units built prior to 1940
DIS - weighted distances to five Boston employment centres
RAD - index of accessibility to radial highways
TAX - full-value property-tax rate per 10,000
PTRATIO - pupil-teacher ratio by town
B - 1000(Bk - 0.63)^2 where Bk is the proportion of blacks by town
LSTAT - % lower status of the population
MEDV - Median value of owner-occupied homes in $1000's, Target Value (Y label)

4.1.1 Correlation among Features

Here we create features correlation heatmap to see How each feature is correlated to the target variable MEDV

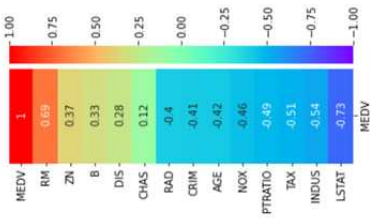

Figure 2: Feature correlation Heatmap

From the correlation heatmap we can infer the importance of features and some insights for example:

- RM: For a higher RM, one would expect to observe a higher MEDV.This is because more rooms would imply more space hence the cost.
- LSTAT: For a higher LSTAT, one would expect to observe a lower MEDV. Generally, an area with more so called 'lower class' citizens would have lower demand, hence lower prices.
- PTRATIO: The prices of houses around public schools are generally lower than those around private schools because there would be a lower teacher-to-student ratio in public schools resulting in less attention dedicated to each student that may impair their performance. Hence one would expect a lower price given a high student-to-teacher ratio due to a lower demand for houses in such areas.

We standardize the data by scaling it and removing the mean and variance to address the problem of the data being on different scales.

4.1.2 Principal Component Analysis (PCA)

Principal component analysis is done to find the least number of features required to have a good model hence we will reduce number of features and determine the approximate number of qubits required.

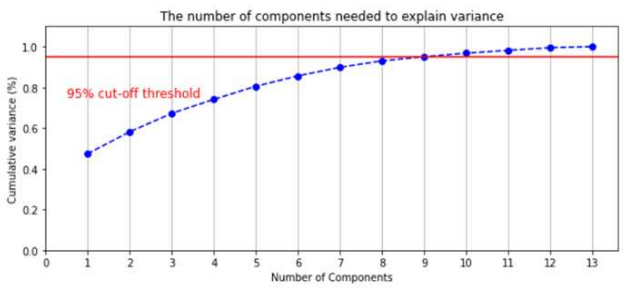

Figure 3: Principal Component Analysis (PCA)

Here we can see that to get at-least 95% of variance explained we need 9 principal components.

Now since we have our data pre-processed and ready, in the following sections we will employ various different types of Quantum Hybrid & Classical Neural networks to the dataset for price predictions and compare the performance of all the approaches.

4.2 Quantum Classical Hybrid Neural Network Architecture

First step is to create a quantum device, after trying various simulators available in pennylane we selected the *default.qubit.tf* device which is based on tensorflow and has intrinsic support for tensor calculus & differentiation it is faster than default and many other devices.Then create the pennylane qnode using qnode decorator.

Qnode denotes the quantum circuit annotated by the device on which it should execute. This functionality provides immense flexibility in defining quantum circuits allowing multiple devices to be used in parallel for multiple qnodes Fig. 4, also the device decorator has many other attributes like 'parallel' as applicable by the supporting device.In qnode a quantum function is defined which is basically the quantum circuit, we can use various kinds of operations, embedding, circuit templates in this function, we can also define our own gate-based operations & custom circuit, where in the function takes inputs & weights as parameters.

In the qnode first step is to map Qubits to features. For which AngleEmbedding from pennylane is used. It encodes N features into rotation angles of n qubits where $N \neq n$. AngleEmbedding chosen over AmplitutedEmbedding because the dataset is fairly large with 500+ samples and so is the number of features which would be very complex and thus difficult to encode & run on a simulator or five qubit IBMQ QPU. Wherein AngleEmbedding provides much more complexity & flexibility than the simpler BasisEmbedding and relatively easy to use compared to AmplitutdEmbedding.

```
@qml.qnode(dev)
def qnn_circuit(inputs, weights):
    qml.templates.AngleEmbedding(inputs, wires = range(n_features))
    qml.templates.StronglyEntanglingLayers(weights, wires = range(n_features))
    return [qml.expval(qml.PauliZ(i)) for i in range(n_features)]
```

Figure 4: Qnode defining the QNN to be executed on device dev.

For QNN layer StronglyEntanglingLayers template is used, it will create a circuit with layers consisting of single-qubit rotations and entanglers, inspired by the circuit-centric classifier design described in Schuld et al [1] It allows to train Quantum Layer using features as angle parameters and is better suited for predictions involving continuous variables.

Keras is used to create the classical neural network architecture, The first layer clayer_in defines the input layer & clayer_out for predicted output Dense Layer with linear activation is used. The Complete neural network model has the quantum circuit layer sandwiched between the input and output classical NN layers i.e. [clayer_in, qlayer, clayer_out]

After creating the qml.qnn.KerasLayer, which simply wraps a QNode into a layer that's compatible with TensorFlow and Keras, TensorFlow is able to clas-

sically optimize the network. The gradient for the quantum part of the network is supplied by the QNode and is calculated by different means depending on the device used (in the strawberryfields.fock case it's calculated by finite differences), while all other gradients are calculated classically by TensorFlow for example using SGD.

Since linear regression is the problem at hand, selected loss type is mean squared error MSE, it squares the difference before summing them all instead of using the absolute value.

Following is a diagramatic presentation of the entire neural network (input is a 9-dimension feature vector)

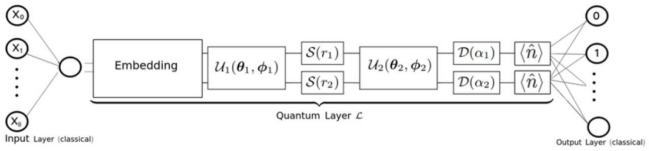

Figure 5: Quantum classical Hybrid QNN with one Quantum circuit layer sandwiched between two classical input & output NN layers

4.2.1 Classical Neural Network Architecture

Classical Neural Network is also created using tensorflow and Keras, the architecture of the NN contains three layers similar to the QNN-Hybrid approach the input layer has number of features as inputs & one hidden layer followed by an output layer, Same SGD optimizer is used to reduce the loss. Most of the architecture & parameters have been kept same as the Hybrid-QNN for fair comparison

4.2.2 Photonic Quantum Classical Hybrid Neural Network Architecture

Architecturally this QNN similar to the Hybrid-QNN defined earlier, the key difference is that quantum circuit is built to be executed on a photonic QPU using strawberryfields library.The Qnode (quantum circuit)is created by using DisplacementEmbedding for feature encoding and CVNeuralNetLayers as the photonic quantum neural network layer for continuous variables.

```
@qml.qnode(dev_photonic)
def qnn_circuit_ph(inputs, w0, w1, w2, w3, w4, w5, w6, w7, w8, w9, w10):
    qml.templates.DisplacementEmbedding(inputs, wires=range(n_features))
    qml.templates.CVNeuralNetLayers(w0, w1, w2, w3, w4, w5, w6, w7, w8, w9, w10, wires=range(n_features))
    return [qml.expval(qml.X(wires=i)) for i in range(n_features)]
```

Figure 6: Photonic QNN Qnode

DisplacementEmbedding Encodes N features into the displacement amplitudes r or phases Φ of M modes(qubits), where $N \leq M$. CVNeuralNetLayers is A sequence of layers of a continuous-variable quantum neural network, as specified in [1]. The layer consists of interferometers, displacement and squeezing gates mimicking the linear transformation of a neural network in the x-basis of the quantum system, and uses a Kerr gate to introduce a 'quantum' nonlinearity.

Similar to earlier QNN the photonic quantum classical hybrid neural network model has the quantum circuit layer sandwiched between the input and output classical NN layers [clayer_in, qlayer, clayer_out]. Here too Stochastic Gradient Descent Optimizer is used.

Essentially three neural network models were trained to predict the house prices, configuration details of these architectures are listed here:

	Hybrid QNN	**Classical NN**	**Photonic QNN**
Features used	9	9	3
Epochs	25	25	25
Learning Rate	0.08	0.08	0.08
Batch Size	5	5	5
Shots	1024	NA	1024
NN layers	3	3	3

5 Results

A machine with core i7 CPU & 32gb memory was used to conduct the simulations. In the above table we see that the Photonic QNN is using only three feature this is because the photonic library Strawberry Fields Fock backend is very computationally intensive, the memory required for simulation scales like D^N, for D dimensions and N wires/qubits. While CV neural nets are conceptually very compelling, they are extremely costly to simulate (because of their infinite-dimensional Hilbert spaces). At 6 features the model error-ed out giving out of memory and at 5 features the model went on training for over 10 hours and then timed out thus for the simulation purpose finally 3 features were used for training, which took around two hours to train & as expected the model pretty much did not learn anything.

5.1 Investigating the Loss Landscape

Figure. 7 illustrates the loss decay comparison with number of epochs, lower the loss higher is the prediction accuracy of the model. We can see that. First Quantum Neural network model performs slightly better than the simple classical neural network with the same settings & configurations though it took about three times the time to train compared to the classical NN in future with large scale Quantum Computers this could be explored on actual QPUs, the accuracy of the simple NN can be made better with changing the Neural network architecture but for fair comparability the architecture was kpet almost similar. We can see that the QNN is already performing slightly better, this could further be tweaked explored

by using different settings like encodings or different ansatz.

Photonic QNN even with very less number of features is also able to perform with near comparable performance demonstrating similar loss decay & bearing in mind that these are simplest QNNs, In future when higher end Quantum devices are available more complex robust QNN-Hybrid architectures could provide manifold advantages.

5.2 Comparing Actual vs Predicted

5.2.1 Comparing Prices predicted by Hybrid-QNN

Here we can see in Fig. 8 that the model has generalized fairly well and the predicted prices are having closer variability to the predicted prices, these are results from a very simple QNN trained on a simulator which could be made much better with more complex larger QNN architectures.

5.2.2 Comparing Prices predicted by Classical NN

Here we can see in Fig. 9 that the model has not generalized as well as the Hybrid-QNN model, well of course the model architecture could be made much better with current ML techniques & frameworks to make the predictions most accurate but with comparable settings with the QNN, the QNN generalizes fair amount better.

5.2.3 Comparing Prices predicted by Photonic QNN

In this case we notice that the model has not learnt anything at all ref Fig. 10, this is primarily because of the fact that the number of features used to train the model was very small due to memory and compute power limitations. as stated earlier with the availability of higher end quantum devices this CVQNN architecture could be improved far more. Now with cloud availability of Xanadu's Borealis much better experiments can be tried.

5.3 Discussion & Future Work

From the results and comparisons with fair confidence we can deduce that even with the simplest settings and configurations the quantum parameterized circuits or the so-called Quantum Neural networks generalised fairly well and as the technology and research matures quantum hybrid approaches show promising results over simple classical approaches in many areas especially where there is involvement of classically intractable problems for example Finance & quantum chemistry wherein plenty of the problems have extremely high dimensionality.

Qubit-based quantum computers have a drawback that they are not completely continuous, since the measurements of qubit-based circuits are mostly discrete. Therefore they are not very suited to continuous variable problems [31]. On the other hand the quantum computing architecture which is more suitable to continuous-variable (CV) problems is photonic QPUs where-in the Quantum information is not encoded in qubits, but is encoded in quantum states of continuous spectrum fields like electromagnetic waves.

Therefore as Future Research, different photonic CV QNN ansatz & approaches can be explored like some described in [1] with actual photonic quantum hardware's like Xanadu's Borealis.

6 Acknowledgment

P. J. is grateful to A. G. G. for his guidance & suggestions & to *Universidad Politecnica de Madrid* for supporting this research. The authors acknowledge the use of libraries Pennylane & Strawberryfield by Xanadu and Tensorflow & Keras by Google.

References

[1] Nathan Killoran, Thomas R. Bromley, Juan Miguel Arrazola, Maria Schuld, Nicolás Quesada, and Seth Lloyd Continuous-variable quantum neural networks 2019

[2] Michael A. Nielsen. Neural Networks and Deep Learning. misc. 2018. url: http://neuralnetworksanddeeplearning.com

[3] International Journal of Computer Vision https://www.springer.com/journal/11263

[4] Anumanchipalli, G.K., Chartier, J. Chang, E.F. Speech synthesis from neural decoding of spoken sentences. Nature 568, 493–498 (2019). https://doi.org/10.1038/s41586-019-1119-1

[5] MuZero by deepmind: Mastering Go, chess, shogi and Atari without rules https://www.deepmind.com/blog/muzero-mastering-go-chess-shogi-and-atari-without-rules

[6] Richard P.Feynman Simulating Physics with Computers https:/people.eecs.berkeley.edu/ christos/classics/Feynman.pdf

[7] Michael A. Nielsen and Isaac L. Chuang. Quantum Computation and Quantum Information: 10th Anniversary Edition. 10th. USA: Cambridge University Press, 2011.

[8] A. Einstein, B. Podolsky, and N. Rosen Can Quantum-Mechanical Description of Physical Reality Be Considered Complete? paper

[9] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," Proceedings 35th Annual Symposium on Foundations of Computer Science, 1994, pp. 124-134, doi: 10.1109/SFCS.1994.365700.

[10] Maria Schuld and Francesco Petruccione. Supervised Learning with Quantum Computers. 1st. Springer Publishing Company, Incorporated, 2018.

[11] Marcello Benedetti et al. "Parameterized quantum circuits as machine learning models." In: Quantum Science and Technology 4.4 (Nov. 2019), p. 043001. issn: 2058-9565. doi: 10.1088/2058-9565/ab4eb5.

[12] Mikko Möttönen et al. "Transformation of quantum states using uniformly controlled rotations." In: Quantum Information Computation 5 (Sept. 2005),pp. 467–473. doi: 10.26421/QIC5.6-5

[13] John Preskill, Quantum Computing in the NISQ era and beyond https://quantum-journal.org/papers/q-2018-08-06-79/

[14] Farhi E, Neven H (2018) Classification with quantum neural networks on near term processors. arXiv preprint arXiv:180206002

[15] Amira Abbas et al. The power of quantum neural networks. 2020. arXiv:2011.00027 [quant-ph]

[16] Penrose, R. (1994) Shadows of the Mind. A search for the missing science of consciousness. Oxford University Press, New York, Oxford.

[17] Seth Lloyd et al. Quantum embeddings for machine learning. 2020. arXiv:2001.03622 [quant-ph]

[18] Barren plateaus in quantum neural network training landscapes Mclean et al

[19] Andrea Skolik et al. Layerwise learning for quantum neural networks. 2020.arXiv: 2006.14904 [quant-ph]

[20] Sim S, Johnson PD, Aspuru-Guzik A (2019) Expressibility and entangling capability of parameterized quantum circuit Advanced Quantum Technologies 2(12):1900,070

[21] Zyczkowski K, Sommers HJ (2005) Average fidelity between random quantum states. Physical Review A 71(3):032,313

[22] Kullback S, Leibler RA (1951) On information and sufficiency. The annals of mathematical statistics 22(1):79–86

[23] Meyer DA, Wallach NR (2002) Global entanglement in multiparticle systems. Journal of Mathematical Physics 43(9):4273–4278

[24] Renatas Kizys et al, A simheuristic algorithm for the portfolio optimization problem with random returns and noisy covariance

[25] Cai, Mc., Deng, X. (2008). Arbitrage in Frictional Foreign Exchange Market. In: Kao, MY. (eds) Encyclopedia of Algorithms. https://doi.org/10.1007/978-0-387-30162-4_33

[26] Marco Pistoia, et al, Quantum Machine Learning for Finance, Future Lab for Applied Research and Engineering, JPMorgan Chase Bank, N.A. 2021

[27] Nathan killoran, Maria Schuld et al, PennyLane Automatic differentiation of hybrid quantum-classical computations 2018.arXiv: 1811.04968 [quant-ph] https://arxiv.org/abs/1811.04968v4

[28] Nathan killoran, Josh Izaac et al, Strawberry Fields: A Software Platform for Photonic Quantum Computing 2019.arXiv: 1804.03159 [quant-ph]

[29] Verdon G, Neven et al , TensorFlow Quantum: A Software Framework for Quantum Machine Learning 2020.arXiv: 2003.02989 [quant-ph]

[30] Keras Chollet, François, 2015, https://github.com/fchollet/keras

[31] A Perdomo-Ortiz, M. Benedetti, J. Realpe-Gómez, and R.Biswas, Quantum Sci. Technol. 3, 030502 (2018).

A Charts from the Results

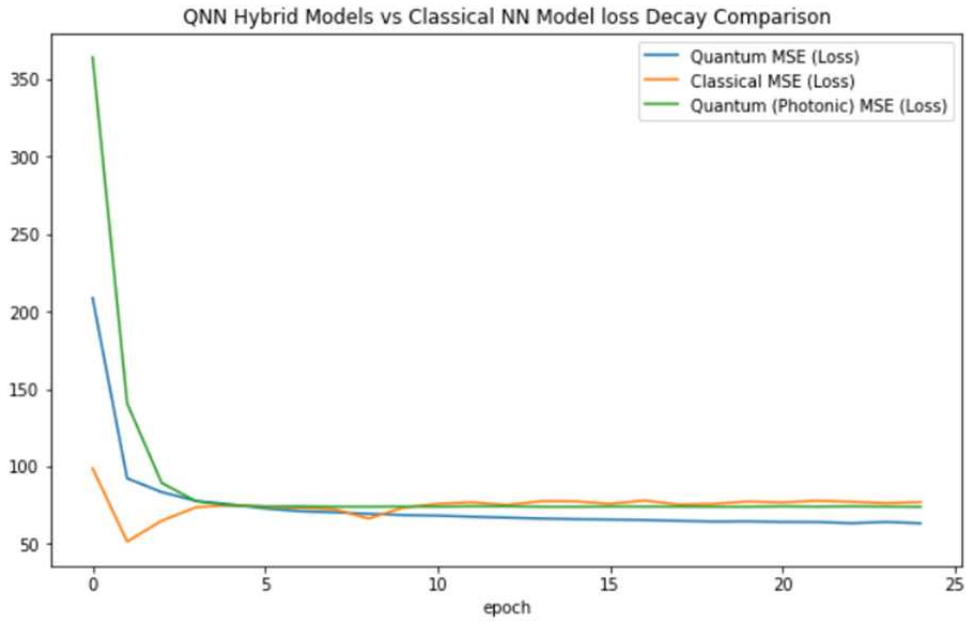

Figure 7: Loss Landscape

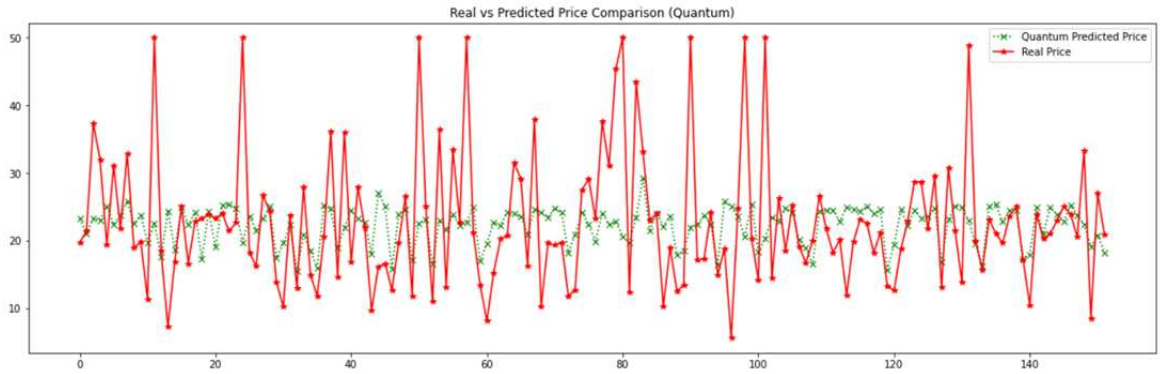

Figure 8: Comparing Prices predicted by Hybrid-QNN model

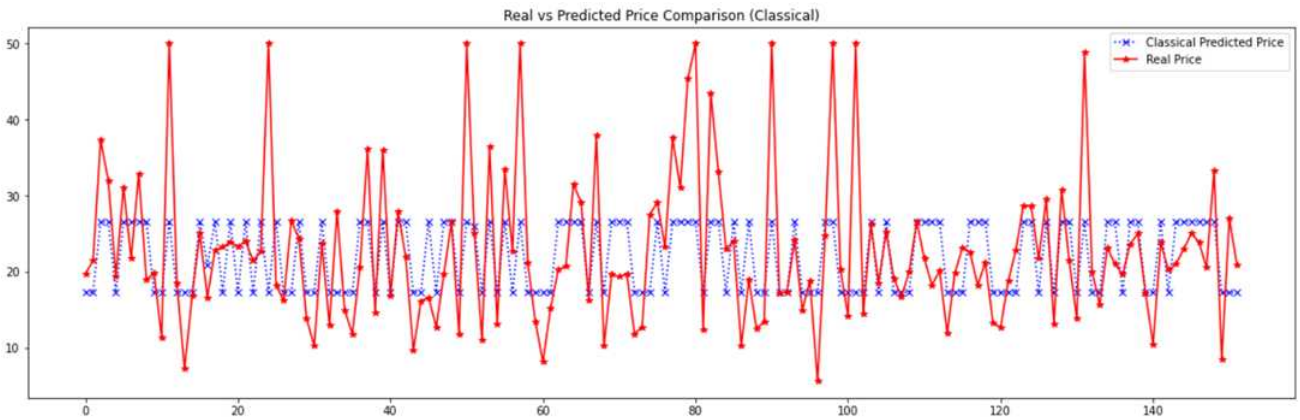

Figure 9: Comparing Prices predicted by Classical NN model

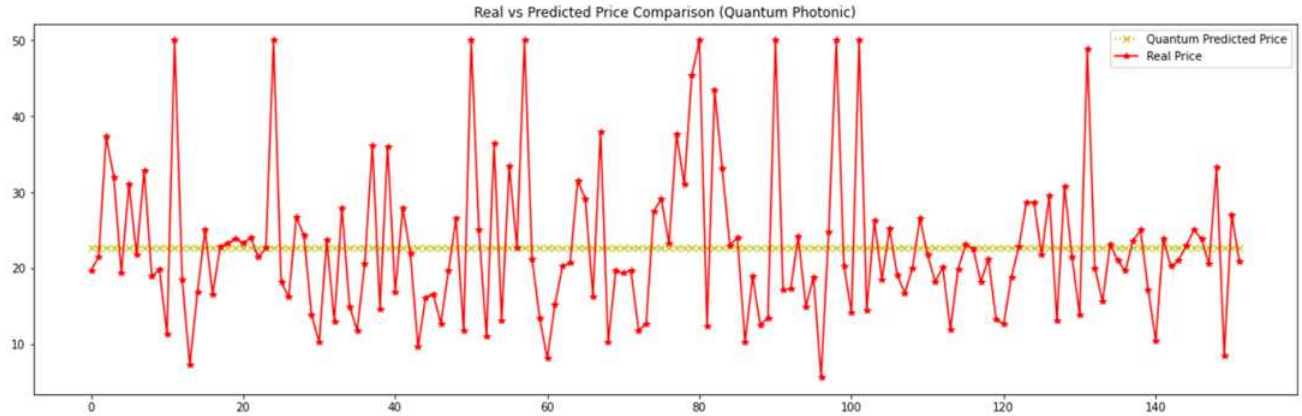

Figure 10: Comparing Prices predicted by Photonic QNN model